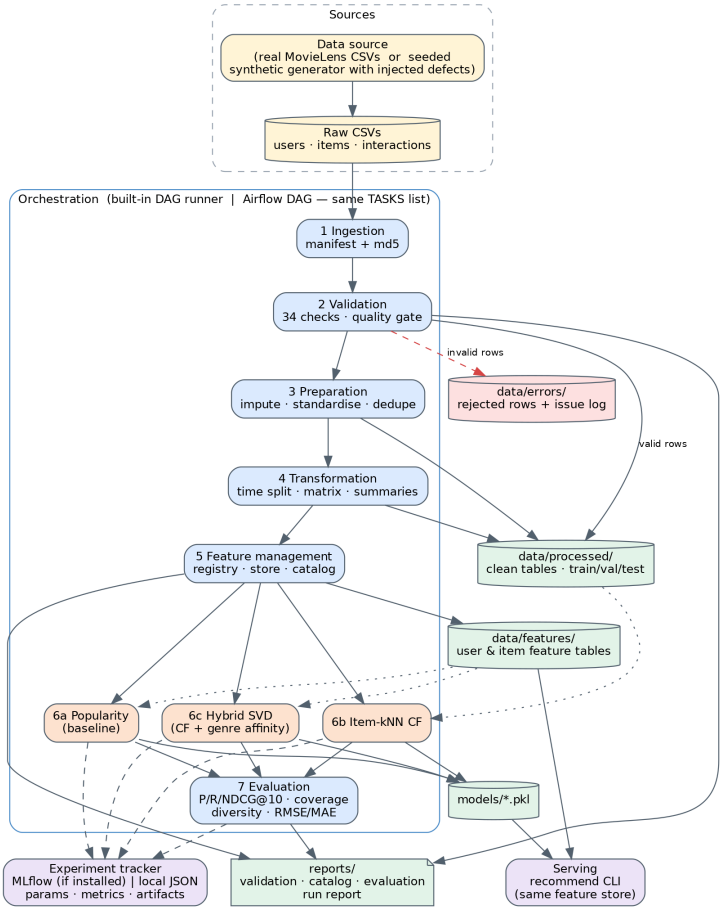
End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning – Assignment 2 · final report

1. Overview and design decisions

The pipeline recommends movies and runs as 10 dependency-ordered tasks: ingestion, validation, preparation, transformation, feature management, training of the models, evaluation, with orchestration over all of them. Every stage reads and writes files (Parquet/JSON/NPZ/pickle) under *data/*, *models/* and *reports/*, so each stage can be re-run, inspected and tested on its own. The whole run is deterministic: two consecutive runs with regenerated data produced byte-identical datasets and identical metrics.



- **Data.** A seeded MovieLens-style synthetic dataset (2,000 users, 900 movies, ~60k ratings, 2023-2025) in which both popularity and personal taste drive viewing and rating, with realistic injected defects (missing values, invalid ratings, bad/future/ancient timestamps, orphan keys, duplicates, mixed formats). Results are therefore *not* comparable with published MovieLens numbers; the pipeline also runs on real MovieLens (see README).
- **Validation before cleaning.** 34 checks (great_expectations 1.24.1) run on the raw string data. *Errors* quarantine a record into *data/errors/* with the reason and original line number; *warnings* (e.g. a missing value) keep the record and are repaired in preparation. A quality gate (reject rate > 15%) fails the run, and downstream tasks are marked *upstream_failed*; I verified this behaviour on a copy of the project with a stricter threshold.
- **Preparation / transformation.** Imputation (median age: 42 rows), standardised gender, country and genres, and one rating per (user, item) (latest kept; 442 older duplicates dropped). A chronological split avoids leakage: train 47,213 / test 11,804 rows, with a validation slice used only for tuning. Derived datasets (user activity summary, item popularity, sparse user-item matrix) come from train only.
- **Feature management.** 12 user, 8 item and 3 user-item feature definitions live in one registry; the catalog is generated from it and checked against the materialised tables on every run. Features are published to **Feast** (user and item feature views), retrieved for training with point-in-time joins at the split date and served online from the SQLite online store; an online-vs-offline parity check guards against training/serving skew. Features are computed on train only and recency is measured against the split date. While building it I found and fixed an index-alignment bug that shifted genres between items; a unit test now guards it.

2. Models, tools and orchestration

- **Popularity** (baseline): rank by training interaction count, Bayesian rating as tie-break and rating predictor.
- **Item-kNN CF**: cosine item-item similarity on the 'liked' (rating ≥ 4) matrix, top-N neighbours, shrinkage.
- **Hybrid SVD**: PureSVD latent score on liked interactions blended with a content signal (user genre affinity from the feature store), plus a biased-SVD rating head giving RMSE/MAE. Hyper-parameters are chosen by validation NDCG@10 (explicit rank by validation RMSE) and the model is refit on all pre-test data.
- **Surprise SVD**: scikit-surprise's biased SVD on explicit ratings; ranks by predicted rating, so it is strong on RMSE/MAE but typically weaker for top-N.
- **Tools.** Python, pandas, NumPy, SciPy; Great Expectations (validation); Feast (features); Parquet storage; MLflow (tracking); scikit-surprise; Graphviz/matplotlib for figures; pytest. 4 tracked runs. Orchestration: Apache Airflow DAG generated from the shared task list (successful run screenshot included); a built-in DAG runner (parallel training tasks, retries, failure propagation, HTML run report) is also provided.

3. Results

Test protocol: newest 20% of interactions; for each of 1,446 users with history and ≥1 relevant test item (rating ≥ 4), the whole catalogue minus training items is ranked and the top 10 compared with relevant test items. RMSE/MAE cover all test ratings for models with a rating head.

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
popularity	0.0255	0.0673	0.0497	0.0467	0.9057	0.9184	0.7575
item_knn	0.0703	0.1795	0.1438	0.4656	0.5218	n/a	n/a
hybrid_svd	0.0763	0.1921	0.1539	0.5478	0.4425	0.8201	0.6619
surprise_svd	0.0102	0.0233	0.0154	0.1000	0.8543	0.8307	0.6654

Both personalised models beat the popularity baseline on NDCG@10 (item_knn 2.89× and hybrid_svd 3.10× the baseline's). hybrid_svd is best on NDCG@10 (0.1539), ahead of item_knn (0.1438). hybrid_svd has the best recall@10 (0.1921) and hybrid_svd the widest catalogue coverage (54.8%; popularity 4.7%). Rating error is lowest for hybrid_svd (RMSE 0.820, MAE 0.662) against 0.918 for popularity; scikit-surprise SVD predicts ratings well but ranks poorly (NDCG@10 0.015), because it is trained for rating error, not top-N ranking. Genre diversity of the lists: hybrid 0.44 vs popularity 0.91 - personalisation costs some diversity. All comparisons come from one chronological split without confidence intervals, so they are indicative.

Data validation summary

Table	Rows in	Rejected	Reject rate	Rows w/ warnings
users	2010	18	0.90%	71
items	904	4	0.44%	19
interactions	64571	5112	7.92%	1914

5,134 records were quarantined; most frequent error checks: interactions.exact_duplicate_row (1,003); interactions.orphan_user_id (818); interactions.orphan_item_id (651); interactions.missing_rating (632); interactions.rating_out_of_range (508). Rejected users also remove their interactions through the referential check, so the interaction reject rate exceeds the injected rate. Warnings kept and repaired: interactions.duplicate_user_item_pair (1,914 rows); users.missing_age (31 rows); users.missing_country (20 rows); users.invalid_age (12 rows). Built-in runner: 10/10 tasks succeeded in 26.22 s (*reports/figures/successful_run.png*); Airflow run: *reports/figures/airflow_successful_run.png*.

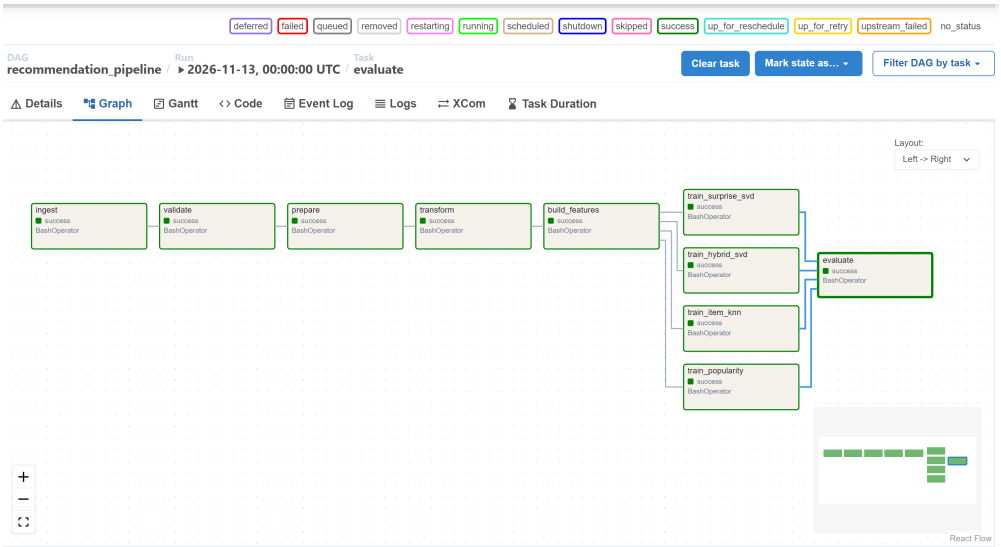


Figure: Apache Airflow graph view of the DAG run - all tasks succeeded.

4. Limitations

- **Data:** synthetic data: absolute scores say little about real users and the generator shapes how easy personalisation is; the real-MovieLens path is documented in the README.
- **Evaluation:** one chronological split, no confidence intervals, no online test; the selected hybrid setting has k_implicit=16 on the edge of the tuning grid (a wider grid might do slightly better); cold-start users get a popularity fallback and are excluded from ranking metrics; the rating head is evaluated for 3 of 4 models (item_knn has none).
- **Scale:** item-item similarity is computed in row chunks and kept as a sparse top-N matrix (it handled 904 movies here), but scoring builds dense users × items blocks in memory; catalogues beyond a few hundred thousand items would need approximate neighbour search and sparse top-N retrieval.